RELATIONSHIP & SYNERGY INFOGRAPHIC

온톨로지 · 지식그래프 · GraphRAG
무엇이 데이터로 흐르고, 무엇이 규칙으로 흐르는가

세 층의 관계 · 규칙 경로 · 환류 · 자기 강화 루프 · **AI 자동 설계와 전문가 검증**

실선 = 데이터(사실) 흐름 · 본류 | 파랑 점선 = 규칙·스키마 제약 | 청록 점선 = 환류(스키마 개정 제안) | 보라 점선 = 자기 강화 루프 | 주황 = 사람(전

01 세 층의 정체와 경계

경쟁 관계가 아니라 한 파이프라인의 세 층입니다. 온톨로지 = T-Box(타입·계약), 지식그래프 = A-Box(인스턴스), GraphRAG = 질의·생성 런타임.

LAYER 1 · 계약

온톨로지 = 설계도

"무엇이 존재할 수 있고, 어떻게 연결되며, 무엇이 필수인가"를 기계가 읽을 수 있는 형태로 정한 계약입니다. 데이터 자체는 담지 않습니다.

담는 것 · 클래스, 관계와 도메인/레인지, 식별자, 제약(필수·카디널리티), 용어집, 출처 규칙

표준 · W3C OWL 2는 "클래스·속성·개체·데이터값"을 갖는 온톨로지 언어이며 형식적 의미를 정의합니다 · W3C SHACL은 그래프가 형태(shapes) 제약을 만족하는지 검증하는 언어입니다

LAYER 2 · 사실

지식그래프 = 서가

계약을 통과한 **실제 사실(노드·엣지)**만 저장합니다. 제안(초안)과 운영(승인된 사실)을 분리해 두는 것이 핵심입니다.

담는 것 · 노드·엣지 + 출처(어디서 왔는가) + 시점(언제 알았는가·언제부터 유효한가) + 신뢰 등급

표준 · W3C PROV-O는 출처 정보를 교환하기 위한 온톨로지입니다 · **제품** · Palantir Foundry 문서에는 "Review ontology proposals", "Ontology branches"처럼 제안 검토·분기 절차가 제품 기능으로 문서화되어 있습니다

LAYER 3 · 런타임

GraphRAG = 사서

질문을 받아 **그래프와 원문**
가지 못하게 묶어 답합니다
질문을 함께 다룹니다.

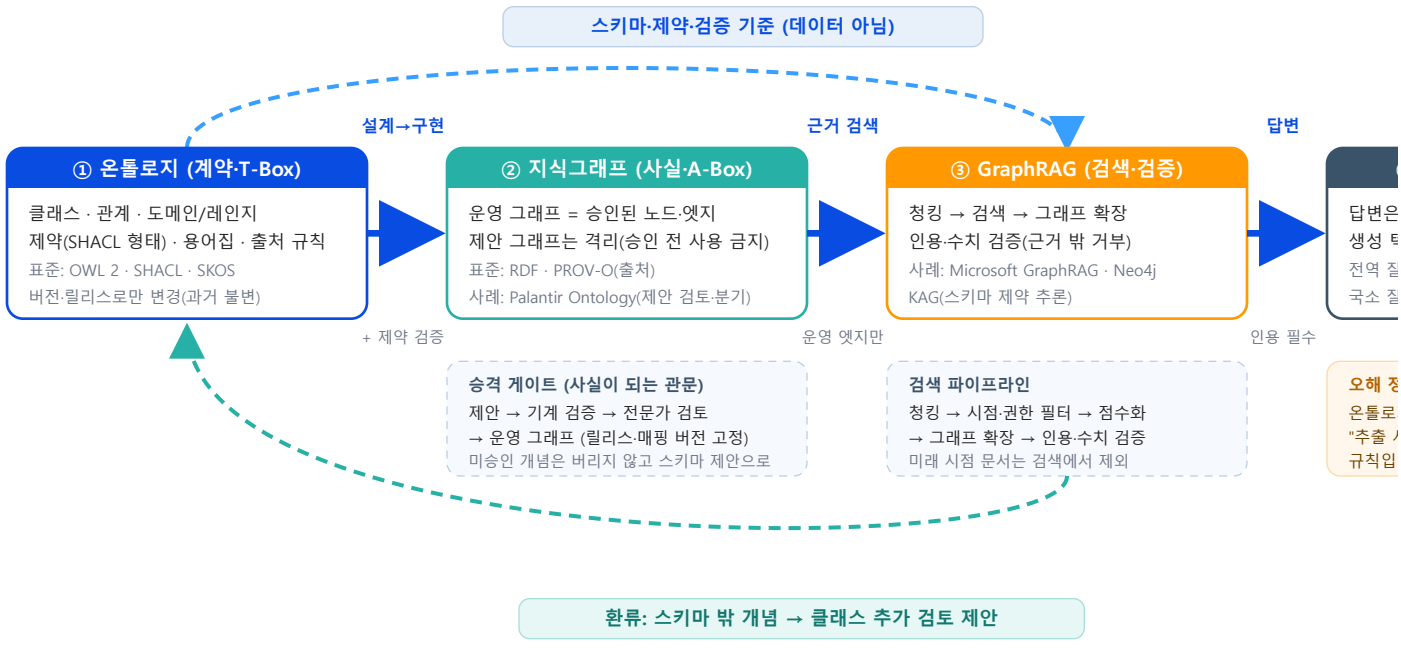
담는 것 · 문서 조각(청크), .
칙, 커뮤니티 요약

연구 · Microsoft GraphRAG는 L
니티 요약을 만들어 전역 질문에
graphrag-toolkit 등 1st-party 구

02 관계도 — 본류 1개 + 보조 2개

굵은 실선이 본류(설계 → 그래프 구현 → 근거 검색 → 답변)입니다. 온톨로지에서 GraphRAG로 가는 점선은 **데이터가 아니라 규칙**이 흐릅니다.

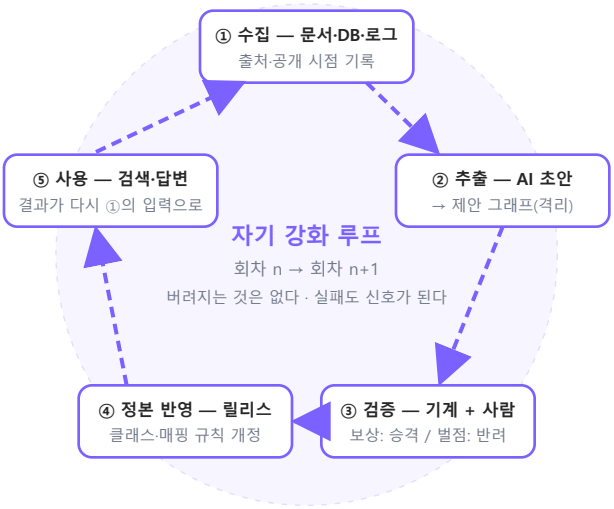
그림 1. 데이터 흐름(굵은 실선) · 규칙·스키마 제약(파랑 점선) · 환류(청록 순환 점선)



03 자기 강화 루프 — 회차를 거듭할수록 설계가 강해지는 원리

환류는 한 번 되돌아오는 화살표가 아닙니다. **매 회차의 성공과 실패가 다음 회차의 스키마·추출 규칙·검색 품질을 바꾸는 누적 학습 고리**입니다.

그림 2. 회차 n의 처리 결과(보상·벌점)가 회차 n+1의 온톨로지·추출 규칙·검색으로 되돌아가는 자기 강화 루프



보상 신호 · 검증 통과 → 승격
성공한 클래스·관계 매핑이 다음 회차 추출의 **정답 예시**로 굳습니다.
벤치마크 · Microsoft GraphRAG의 Auto Prompt Tuning은 도메인에 맞는 추출 프

벌점 신호 · 미매핑·미해결 관계·출처 누락
버려지지 않고 **스키마 변경 제안·추출 규칙 개선 후보**로 축적됩니다.
벤치마크 · Palantir Foundry의 "Review ontology proposals"(온톨로지 제안 검토)

망각 방지 · 계약을 테스트로 고정
회차가 늘어도 검증 규칙이 흐려지지 않게 제약을 자동 검사로 붙잡습니다.
벤치마크 · W3C SHACL(형태 제약 검증) · OWL 2(형식적 의미)

비교 가능성 · 같은 입력 → 같은 결과
입력·릴리스·매핑 버전을 고정하면 회차 간 비교("강해졌는가")가 성립합니다
벤치마크 · 릴리스·매핑 버전을 그래프 산출물에 함께 기록하는 관행(온톨로지 거

사람이 닫는 고리 · 의미와 책임
자동화가 끝까지 닫지 못하는 부분은 **의미 확정과 승인**입니다. AI는 초안·검
문가가 서명합니다.
관련 절 · 04 AI가 설계하고, 전문가가 검증한다

04 AI가 설계하고, 도메인 전문가가 검증한다

과거에는 전문가가 온톨로지를 **손으로 작성**했습니다. 지금은 AI가 문서·DB·로그에서 **초안을 자동 생성**하고, 전문가는 **의미와 업무 규칙을 검증·승인**하는 분업이 성립합니다.

그림 3. AI 자동 초안 → 기계 검증 → 전문가 검증(주황) → 릴리스 → 운영 반영 → 다시 AI 학습 신호로

1 **소스 수집**
문서·DB 스키마·로그·API를 권리·보안 등급과 함께 등록

2 **AI 초안 생성**
클래스·관계·제약·동의어 후보를 대량 추출(LLM·규칙 파서 결합)

3 **기계 검증**
형식·중복·도메인/레인지·SHACL 제약 자동 검사

4 **전문가 검증**
의미·경계·업무 규칙·법규·예외를 판단하고 승인/수정/반려

역할 분담 — 자동화와 책임의 경계

AI(자동)가 잘하는 것	도메인 전문가가 반드시 하는 것
대량 문서에서 후보 클래스·관계 추출	개념의 의미·경계 확정(다의어·동의어 판정)
이름·정의·매핑 **초안** 제시	업무 규칙·법규·회계·안전 기준의 타당성 판단
형식 오류·중복·도메인 위반 자동 점검	예외와 우선순위 결정, 책임 소재 확정
유사 개념·변경 영향 범위 탐색	승인 여부 결정(책임 있는 서명)

검증 게이트 최소 요건 5가지

- **작성자 ≠ 승인자** · AI가 만든 초안을 AI가 승인하지 않는다(교차
- **모든 사실에 출처** · 원문 위치·버전·해시까지 연결한다(PROV-O
- **미승인은 사용 금지** · 제안 평면과 운영 평면을 분리한다
- **릴리스 불변·되돌리기** · 과거 릴리스는 고치지 않고 새 버전으로
- **감사 이력** · 누가·언제·무엇을·왜 승인/반려했는지 남긴다

주의 · "AI가 온톨로지를 자동 설계한다"는 말은 **초안 생성의 자동화**를 뜻합니다. 승인까지 자동화하면 잘못된 개념이 그래프·검색·의사결정 전체로 전파됩니다. 사람 검증 게이트를
것이 안전합니다.

05 두 개씩 짝지어 본 관계

각 쌍은 서로 다른 것을 주고받습니다. 한쪽이 없으면 어떤 힘이 사라지는지 함께 적었습니다.

온톨로지 ↔ 지식그래프

주는 것 클래스·관계 허용 범위, 식별자·필수 필드, 출처·유효기간 규칙

받는 것 실제 데이터에서 매핑되지 않는 개념 → **스키마 개정 후보**(근거 있는 진화)

깨지면 타입 없는 삼중항만 남아 관계 위반·중복·출처 누락을 **탐지할 수단이 없음**

온톨로지 ↔ GraphRAG

주는 것 추출 시 타입 부여 기준, 관계 생성 금지 규칙, 답변 검증 기준

받는 것 문서에서 새로 나온 개념·표현을 대량 공급받아 **개정 후보를 실데이터로 뒷받침**

깨지면 그럴듯한 문장만 만들 뿐 **검증 가능한 사실**을 만들지 못함

지식그래프 ↔ Graph

주는 것 다중 홉 확장 경
보 차단)

받는 것 문서→초안→검
를 계속 키우는 통로)

깨지면 벡터 유사도 검
음

검증 지점 · 도메인/레인지 검사 · SHACL 형태 검사 · 중복 식별자 거부

연결 기술(예시)

표현·제약 RDF 1.1 OWL 2 SHACL SKOS JSON-LD

매핑·적재 R2RML SPARQL CONSTRUCT ETL(Airflow·dbt)

저장·질의 트리플스토어(Jena·GraphDB·Neptune) 속성 그래프(Neo4j) SPARQL·Cypher

검증·추론 pySHACL ShEx OWL 추론기

검증 지점 · 인용 없는 답변 거부 · 근거에 없는 수치 거부 · 스키마 밖 관계 생성 금지

연결 기술(예시)

스키마 주입 JSON Schema(구조화 출력 강제) 프롬프트 템플릿

추출·파이프라인 Microsoft GraphRAG LlamaIndex LangChain Neo4j GraphRAG AWS graphrag-toolkit KAG

검증·정합 pySHACL JSON Schema 검증 엔티티 해소(동일 실체 병합)

자기 강화 Auto Prompt Tuning(도메인 적응)

검증 지점 · 운영 엣지만 순 크 해시로 역추적

연결 기술(예시)

검색 벡터 인덱스(pgvector·FAI 하이브리드 검색

그래프 확장 Cypher SPARQ

추적·감사 청크 해시 PROV-

서빙 API 계층(FastAPI·Django

표. 세 쌍을 실제로 이어 붙이는 구현 기술 스택 (도구 이름은 **대표 예시**, 표준은 W3C 문서로 확인)

연결 지점	무엇을 잇나	표현·표준 계층	처리·매핑 계층	저장·질의·검증 계층
온톨로지 ↔ 지식그래프	스키마 ↔ 인스턴스	RDF 1.1(데이터 모델) · OWL 2(클래스·속성·형식 의미) · SHACL(제약) · SKOS(용어집) · JSON-LD(직렬화)	R2RML(관계형→RDF 매핑) · SPARQL CONSTRUCT(변환 질의) · ETL 오케스트레이션(Airflow·dbt)	트리플스토어(Jena·Gr (Neo4j) · SPARQL/Cyp 기
온톨로지 ↔ GraphRAG	규칙 ↔ 추출·검증	JSON Schema(출력 형식 강제) · 프롬프트 템플릿(스키마를 프롬프트에 주입) · 온톨로지 용어집	Microsoft GraphRAG · LlamaIndex · LangChain · Neo4j GraphRAG · AWS graphrag-toolkit · KAG(스키마 제약 추론)	JSON Schema·pySHA · Auto Prompt Tuning
지식그래프 ↔ GraphRAG	사실 ↔ 검색·근거	청크 해시(문서 조각 식별) · PROV-O(출처) · 인용 식별자 규약	벡터 인덱스(pgvector·FAISS·Milvus) + 키워드(BM25) 하이브리드 · 커뮤니티 탐지·요약(GraphRAG 인덱싱)	Cypher/SPARQL 그래 OpenLineage(데이터

읽는 법 · 왼쪽에서 오른쪽으로 갈수록 "정의 → 변환 → 저장·검증" 순서입니다. **표준 계층을 먼저 고정하고 도구를 나중에 고르면** 저장소·프레임워크를 바꿔도 자산(스키마·매핑·출처 정하면 그래프를 다시 만들어야 합니다. 도구 이름은 공개 문서로 존재를 확인한 대표 예시이며 특정 제품 권장이 아닙니다.

06 셋이 붙을 때 생기는 시너지 8

"효과가 좋아진다"는 주장이 아니라, 어느 장치가 그 효과를 만드는지까지 적었습니다.

시너지	만들어지는 방식	그 효과를 만드는 장치
① **환각 억제**	스키마 검증 × 인용 검증 × 운영 엣지만 확장 — 3중 게이트	SHACL 형태 검사 · 인용 필수 · 그래프 확장 제한
② **재현성·감사**	같은 입력·같은 릴리스면 같은 결과, 승인 이력은 추가만 가능	결정론적 처리·해시 · append-only 감사 로그
③ **시점 정합성**	공개 시점·유효기간을 지켜 미래 정보 사용을 차단	available_at · valid_from/valid_to · 시점 필터
④ **설명가능성**	답변 → 문서 조각 → 출처 → 원문 위치까지 역추적	청크 해시 · PROV-O 계열 출처 필드
⑤ **지식 복리 성장**	문서가 늘면 근거가 늘고, 매핑 실패가 스키마 제안으로 축적	스키마 변경 제안 큐 · 릴리스 버전 관리
⑥ **오염 차단(거버넌스)**	작성자 ≠ 승인자, 제안 평면과 운영 평면 분리	교차 검증 · 제안/운영 저장소 분리
⑦ **비용 통제**	사실 확정은 규칙·검증기가 담당, LLM은 해석·요약에 집중	결정론 파서 · 수치 가드
⑧ **자기 강화(회차 축적)**	회차의 성공·실패가 스키마 제안·추출 프롬프트·매핑 정답으로 누적	보상/벌점 신호 기록 · 자동 프롬프트 조정 · 회차 지표 비교

전역 질문과 국소 질문 · 그래프가 없으면 "이 데이터셋의 주요 주제는?" 같은 **전역(요약형) 질문**에 약합니다. Microsoft GraphRAG는 엔티티 그래프와 커뮤니티 요약을 미리 만들어 데이터셋의 전역 질문에서 기존 RAG 대비 개선을 보고했습니다(논문 초록 기준). KAG는 반대로 수치·시간·전문 규칙처럼 **논리·제약**이 중요한 질문에서 벡터 유사도만으로는 부족하다 제약 결합을 제안합니다.

07 벤치마킹한 사례

공개 표준·제품 문서·논문에서 **직접 확인한 내용**만 적었습니다. 확인하지 못한 수치·내부 자료는 넣지 않았습니다.

구분	사례	공개 자료에서 확인되는 구조적 특징
표준	W3C OWL 2	"클래스·속성·개체·데이터값"을 갖는 온톨로지 언어이며 형식적 의미를 정의
표준	W3C SHACL	그래프가 형태(shapes) 제약을 만족하는지 검증하는 언어

표준	W3C PROV-O	출처(provenance) 정보를 교환하기 위한 온톨로지
제품	Palantir Foundry Ontology	문서 목록에 "Review ontology proposals", "Ontology branches", "Branching the ontology" 등 **제안 검토·분기 절차**가 제품 기능으로 존재
제품·연구	Microsoft GraphRAG	LLM이 엔티티·관계·주장 추출 → 커뮤니티 탐지 → 커뮤니티 요약·보고서 → 임베딩. Auto Prompt Tuning으로 **도메인 적응 추출 프롬프트를 자동 생성**
연구	GraphRAG 논문 (arXiv 2404.16130)	2단계 그래프 인덱스(엔티티 그래프 → 커뮤니티 요약). 약 100만 토큰 규모 **전역 질문**에서 기존 RAG 대비 개선을 보고
연구	KAG (arXiv 2409.13731)	벡터 유사도와 지식 추론의 간극, 수치·시간 관계·전문 규칙에 대한 둔감함을 지적하고 **KG+벡터 결합·스키마 제약 추론**을 제안
제품	Neo4j GraphRAG (Python)	1st-party 패키지로 RAG와 **Knowledge Graph Builder** 가이드를 함께 제공
제품	AWS graphrag-toolkit	"graph-enhanced GenAI applications" 구축용 파이썬 툴킷(오픈소스)
제품	Stardog Voicebox	온톨로지·지식그래프 위에서 질의응답하는 제품 라인으로 공개
제품	Google Knowledge Graph	공식 블로그 "Introducing the Knowledge Graph: things, not strings" — 문자열이 아닌 **사물(엔티티) 중심** 접근을 공개

08 도입 판단 기준과 흔한 실패

기술 선택보다 순서가 결과를 좌우합니다. 아래 최소 단계를 건너뛰면 그래프는 만들어져도 쓰이지 않습니다.

최소 성숙 단계 (권장 순서)

- **1. 질문·결정 목록** · 무엇을 판단하려는지 먼저 적는다(그래프는 수단)
- **2. 용어집 + 최소 스키마** · 핵심 클래스·관계만으로 시작(과설계 금지)
- **3. 출처·시점 연결** · 모든 사실에 "어디서·언제 알았는가"를 붙인다
- **4. 검증 게이트** · 기계 검증 + 전문가 검증을 통과한 것만 운영 반영
- **5. 지표 고정** · 미매핑률·반려율·인용률을 회차마다 기록해 비교한다

흔한 실패 패턴

- **과설계** · 완벽한 온톨로지를 먼저 만들다 데이터 연결이 늦어져
- **스키마 없는 자동 추출** · 타입 없는 삼중항이 쌓여 검증·추론이
- **검증 없는 자동 승격** · 잘못된 개념이 그래프·검색·의사결정 전체
- **시점 무시** · 미래 정보가 섞여 분석 결과가 실제보다 좋아 보인다
- **지표 없음** · "좋아졌다"를 증명하지 못해 확장 근거를 잃는다

요약 · 온톨로지가 **무엇을 사실로 인정할지**를 정하고, 지식그래프가 **그 사실을 축적**하며, GraphRAG가 **그 사실을 근거로만 말하게** 합니다. 여기에 AI의 초안 생성과 전문가의 검증이 추출·검색이 함께 강해지는 자기 강화 구조가 됩니다.

인용 출처(직접 열어 확인) · W3C OWL 2 Overview · W3C SHACL · W3C PROV-O · GraphRAG 논문(arXiv 2404.16130) · KAG 논문(arXiv 2409.13731) · GraphRAG Indexing · GraphRAG Auto Prompt Tuning · · Neo4j GraphRAG (Python) · Palantir Foundry Ontology 문서 · Stardog Voicebox · Google Knowledge Graph 발표 · RDF 1.1 · R2RML · SPARQL 1.1 · JSON-LD 1.1 · SKOS · pySHACL · pgvector · Milvus · Cyp

방법 · 위 공개 문서를 직접 열어 확인한 사실만 인용했고, 요약·종합(그림의 구조와 시너지 정리)은 본 문서의 해석입니다. 확인하지 못한 성능 수치·고객 사례·내부 자료는 넣지 않았습니다. 그림의 시각 규칙(색 사람 검증)은 문서 인포그래픽 스킬의 토큰을 적용했습니다.

파일 docs/ontology-kg-graphrag-infographic.html · docs/ontology-kg-graphrag-infographic.png · docs/ontology-kg-graphrag-infographic.pdf